

Calcolo Iterativo Sparse Matrix-Vector su Matrici Sparse Evolutive

Corso SPM a.a. 25/26

Gioele Modica

Corso SPM a.a. 25/26 - Gioele Modica

1. Problema e Algoritmo

Il calcolo esegue 500 iterazioni di prodotto matrice-vettore sparso (SpMV) su una matrice quadrata CSR A in $\mathbb{R}^{N \times N}$. Ogni EPOCH_LEN = 25 iterazioni, la matrice subisce uno shift circolare delle righe: la riga logica i legge la riga fisica $src = (i + n * row_shift) \% n$. La matrice non viene mai modificata fisicamente; cambia solo l'intero `row_shift`, che aumenta di un valore fisso `shift_rows` ad ogni confine di epoch (20 transizioni totali). Ogni iterazione normalizza il vettore risultante (norma L2), poi scambia i buffer di input e output. Dopo 500 iterazioni il quoziente di Rayleigh $x^T(Ax)$ approssima l'autovalore dominante.

Carico irregolare. In modalità `-m irregular` circa il 12.5 % delle righe (le prime $n/10$) contiene 40-160x più elementi non-zero rispetto alla coda. Questo causa uno sbilanciamento del carico molto severo nelle partizioni statiche ed è la sfida principale della parallelizzazione.

Riferimento sequenziale ($n = 500\,000$, $nz = 20\,000\,000$, `-m irregular`): **52.97 s** su un core fisico.
Riferimento large-scale ($n = 2\,000\,000$, $nz = 80\,000\,000$): **217.93 s**.

2. Strategie di Implementazione

2.1 C++ Threads (`std::thread` + `std::barrier`)

T thread worker vengono creati una sola volta prima del loop e rimangono attivi per tutte le 500 iterazioni. Il meccanismo di sincronizzazione centrale è una singola istanza di `std::barrier<Fn>` con una *completion function* che esegue serialmente ad ogni arrivo alla barriera:

```
completion della barriera (eseguita una volta per iterazione, un solo thread):
1. aggiornamento epoch: row_shift = (row_shift + shift_rows) % n
2. riduzione norma: somma partial_norms[0..T-1]
3. normalizza y: y[i] *= 1/sqrt(total) (O(n), ~2.5 % del costo SpMV)
4. swap(x, y): scambio puntatori, O(1)
5. reset contatore atomico per l'iterazione successiva
```

Questo design garantisce esattamente **una barriera per iterazione**, rispettando il vincolo specificato nella consegna.

Accumulatori thread-local paddingati a 64 byte (`alignas(64) PaddedDouble`) per evitare false sharing sull'array delle norme parziali.

Partizionamento statico (NNZ-bilanciato). I boundary vengono calcolati una sola volta all'avvio tramite ricerca binaria su `row_ptr`, in modo che ogni thread possieda circa $total_nnz / T$ non-zero delle righe fisiche CSR. Il problema critico è che i thread iterano su righe *logiche* e accedono alle righe fisiche tramite `row_shift`. Ad ogni epoch `row_shift` cambia, ruotando il mapping logico->fisico e sbilanciando continuamente il carico. Dopo ogni epoch le righe fisiche "pesanti" (prime $n/10$) vengono mappate a un range logico diverso, che può appartenere interamente a un solo thread. Il bilanciamento statico è corretto solo all'epoch iniziale (`row_shift = 0`).

Scheduling dinamico (work-stealing atomico). Un contatore `std::atomic<size_t> next_chunk` viene azzerato ad ogni iterazione. I thread chiamano `fetch_add(K, relaxed)` per reclamare chunk di K righe logiche. Qualunque thread che finisce il proprio chunk preleva immediatamente il successivo, eliminando ogni periodo di attesa. Questo assorbe completamente lo sbilanciamento indotto dagli epoch al costo di un `LOCK XADD` per chunk.

Granularità dell'unità di lavoro (C++ threads, modalità dinamica). Il costo di prelevare un chunk è una singola istruzione `LOCK XADD` (~3-5 ns su x86), senza allocazione heap né code di scheduling. Per $n=500$

000, $K=64$, $T=16$: operazioni atomiche totali per run = $(n/K) \times 500 \text{ iter} = 3\,906\,000$, con un overhead massimo di $3.9M \times 5 \text{ ns} \sim 0.02 \text{ s}$ (0.3 % di 5.77 s totali). Di conseguenza, la versione dinamica è **insensibile a K** nell'intervallo pratico K in [16, 1024]: a $K=64$ raggiunge già 5.77 s con $T=16$, entro il 20 % dall'ottimo di OMP Tasks a $K=1024$ (4.83 s). Il netto contrasto con OMP Tasks - che degrada da 4.83 s ($K=1024$) a 102.1 s ($K=16$) - conferma che il bottleneck in OMP Tasks è il loop di creazione task serializzato, non la granularità del calcolo. Il contatore atomico scambia il controllo fine dello scheduling con overhead quasi nullo per chunk.

2.2 OpenMP Task-Based

La regione parallela copre tutte le 500 iterazioni. Ogni iterazione usa due blocchi `#pragma omp single`, ciascuno con una barriera implicita:

- **Primo single**: aggiorna opzionalmente `row_shift`, poi crea n/K task (uno per ogni chunk di K righe logiche). Ogni task ha `firstprivate(cs, ce)` per catturare il range di righe al momento della creazione; senza questa clausola la variabile di loop entrerebbe in race condition con l'esecuzione del task. I worker iniziano immediatamente a consumare dalla coda dei task mentre il master li sta ancora creando. `#pragma omp taskwait` dentro il `single` garantisce che tutti i task terminino prima che la barriera implicita venga attraversata.
- **Secondo single**: riduce le norme parziali, normalizza y, scambia i puntatori, resetta gli accumulatori.

Trade-off della granularità dei task. La creazione dei task è serializzata in un solo thread. A K piccolo il produttore non riesce a tenere occupati T consumer: a $T=16$, con $K=64$ vengono creati 7 812 task/iterazione e a $T=32$ il produttore diventa il bottleneck (30.4 s, peggio di $T=8$). Aumentare K riduce l'overhead di creazione: a $K=1024$ con 488 task/iterazione il bottleneck viene eliminato completamente.

2.3 OpenMP Work-Sharing

`#pragma omp for schedule(dynamic, K) reduction(+:norm_sq)` sostituisce la creazione esplicita di task. La clausola `reduction` alloca copie private di `norm_sq` sullo stack di ogni thread (intrinsecamente prive di false sharing). Il meccanismo work-sharing di `omp for` distribuisce i chunk tramite un contatore gestito dal runtime, senza il bottleneck del produttore a singolo thread, garantendo scaling migliore a T grande con K piccolo.

L'aggiornamento dell'epoch viene pre-calcolato alla fine dell'iterazione i dentro il blocco `single` (semanticamente equivalente all'aggiornamento all'inizio dell'iterazione i+1, grazie alla barriera implicita che garantisce l'ordinamento).

2.4 MPI + OpenMP (Ibrida)

Distribuzione della matrice. Il rank 0 genera la matrice completa e la distribuisce tramite `MPI_Scatterv` con boundary NNZ-bilanciati (stesso algoritmo del partizionamento statico C++ threads). Ogni rank mantiene la propria slice di righe fisiche per tutte le 500 iterazioni. Gli offset di `row_ptr` vengono normalizzati a 0 su ogni rank dopo lo scatter.

Evoluzione epoch - AD-002 Opzione A (mapping logico). Le assegnazioni di righe fisiche rimangono fisse. Ogni rank tiene traccia dello scalare `row_shift` condiviso (stessa formula del sequenziale, nessuna comunicazione). Il costo della transizione epoch è $\sim 0 \text{ us}$; non avviene nessun movimento di dati. L'Opzione B (redistribuzione fisica con `MPI_Alltoallv`) è stata scartata perché richiederebbe $20 \times O(n)$ ridistribuzioni su tutti i nodi - proibitivamente costosa a $n = 2 \text{ M}$.

x replicato - AD-003. Ogni rank mantiene il vettore x completo (n double). Poiché ogni riga SpMV può accedere a qualsiasi colonna di x, lo halo exchange è impraticabile senza conoscere il pattern delle colonne non-zero a runtime. La replicazione costa $O(n \times P)$ di memoria ma elimina completamente la comunicazione per gli accessi a x. Il trade-off è valido quando $n \times 8 \text{ byte} \ll \text{RAM disponibile per nodo}$ (16 MB a $n = 2 \text{ M}$ vs. decine di GB per nodo).

Comunicazione per iterazione (due operazioni collettive):

- `MPI_Allreduce(local_sq -> global_sq, MPI_SUM)`: $O(1)$ dati, latenza $O(\log P)$. Fornisce la norma L2 globale; ogni rank normalizza la propria slice locale di `y` in modo indipendente.
- `MPI_Allgatherv(y_local -> phys_buf) + cyclic_copy`: $O(n)$ dati per iterazione. Ogni rank contribuisce la propria slice normalizzata di `y`; `cyclic_copy` riassembla `x` tenendo conto di `row_shift` tramite due chiamate `memcpy` invece di `n` divisioni modulari.

SpMV intra-rank usa `#pragma omp parallel for schedule(dynamic, K) reduction(+:local_sq)` con `T` thread per rank, gestendo lo sbilanciamento del carico all'interno del rank sull'input irregolare. Le chiamate MPI vengono effettuate solo dal thread master (`MPI_THREAD_FUNNELED`).

Ottimizzazione: row_shift assente nel loop caldo. L'implementazione MPI lavora interamente nello spazio fisico durante lo SpMV: ogni rank calcola `y_local[j] = A_local[j,:] · x` per le proprie righe fisiche senza mai eseguire l'operazione `(i + n - row_shift) % n` nel loop interno. Il mapping logico viene applicato una sola volta, in modo vettorizzato, tramite `cyclic_copy` al momento del riassemblaggio di `x` (due `memcpy` contigue). Questo elimina una divisione modulare per ogni riga nella sezione più costosa del codice - circa n operazioni $\% n$ per iterazione risparmiate - migliorando il throughput su matrici large ($n=2M \rightarrow 2M$ divisioni modulari evitate per iterazione).

3. Verifica della Correttezza

Tutte le implementazioni sono state verificate rispetto al riferimento sequenziale con due criteri:

- **Differenza relativa del valore di Rayleigh** $< 1 \times 10^{-10}$
- **Diff elemento per elemento** (one-liner awk, soglia 1×10^{-10} , output vuoto = PASS):

```
awk 'NR==FNR{a[NR]=$1;next} {d=a[FNR]-$1; if(d<0)d=-d; if(d>1e-10) print NR, d}' \
/tmp/seq.dump /tmp/par.dump
```

Valori di Rayleigh misurati ($n = 500\,000$, $nz = 20\,000\,000$, `-m irregular`, `seed = 111`):

Implementazione	rayleigh	\	?\	^	refl	
Sequenziale (T=1, riferimento)	4.223260535478325	-				
Threads dinamico T=4	4.223260535478293	7.6×10^{-15}				
Threads dinamico T=16	4.223260535478219	2.5×10^{-14}				
OMP Tasks K=1024 T=16	4.223260535478210	2.7×10^{-14}				
OMP Work-Sharing T=16	4.223260535478214	2.6×10^{-14}				

MPI+OpenMP ($n = 2\,000\,000$, $nz = 80\,000\,000$, `-m irregular`):

Configurazione	rayleigh	\	?\	^	refl	
Sequenziale (riferimento)	4.223544292121269	-				
MPI np=4 T=4 (1 nodo)	4.223544292121007	6.2×10^{-14}				
MPI np=8 T=4 (2 nodi)	4.223544292121087	4.3×10^{-14}				

Tutti gli scostamenti sono nell'intervallo 10^{-14} - 10^{-13} , quattro ordini di grandezza al di sotto della soglia 1×10^{-10} . Queste differenze sono spiegate dalla non-associatività dell'aritmetica floating-point: le riduzioni parallele accumulano somme parziali in un ordine diverso rispetto al riferimento sequenziale. Il campo `checksum` (hash XOR di `x`) può differire per la stessa ragione e non è un criterio di correttezza.

4. Performance Single-Node

Configurazione: nodo `spmcluster` (16 core fisici, 32 logici con HT), $n = 500\,000$, $nz = 20\,000\,000$, `-m irregular`. Baseline: sequenziale = 52.97 s. Tutti i tempi sono mediane su 3 run.

4.1 Partizionamento Statico vs Dinamico (C++ Threads)

T	Statico (s)	Speedup	Dinamico (s)	Speedup	Efficienza (din)
1	52.97	1.00	53.14	1.00	1.000
2	48.27	1.10	28.10	1.89	0.946
4	43.27	1.22	16.16	3.29	0.822
8	36.45	1.45	9.27	5.73	0.716
16	29.85	1.78	5.77	9.21	0.576
32	23.78	2.23	5.25	10.13	0.317

Il partizionamento statico fallisce nonostante il bilanciamento NNZ. La partizione viene calcolata una sola volta all'avvio sulle righe fisiche CSR. Ogni thread itera su un range fisso di righe *logiche*, accedendo alle righe fisiche attraverso il *row_shift* corrente. Ad ogni transizione di epoch il mapping logico->fisico ruota: il range logico che accedeva alla coda leggera (basso NNZ) ora accede alla testa pesante e viceversa. Nel corso delle 20 transizioni di epoch il lavoro per thread viene campionato uniformemente su tutte le zone di peso, ma in qualsiasi epoch dato un solo thread detiene quasi tutte le righe pesanti - causando uno sbilanciamento $O(T)$. Lo speedup medio di 1.78x a $T=16$ riflette questo sbilanciamento mediato sugli epoch.

Lo scheduling dinamico assorbe completamente lo sbilanciamento. Poiché ogni thread preleva immediatamente il chunk disponibile al termine del proprio, i punti caldi indotti dagli epoch vengono distribuiti sul pool di thread all'interno di una singola iterazione. Lo speedup di 9.2x a $T=16$ è vicino all'ideale su 16 core fisici, confermando che il calcolo domina una volta rimosso l'overhead di scheduling. L'efficienza cala a $T=32$ (HT) poiché due thread logici condividono una stessa unità di esecuzione.

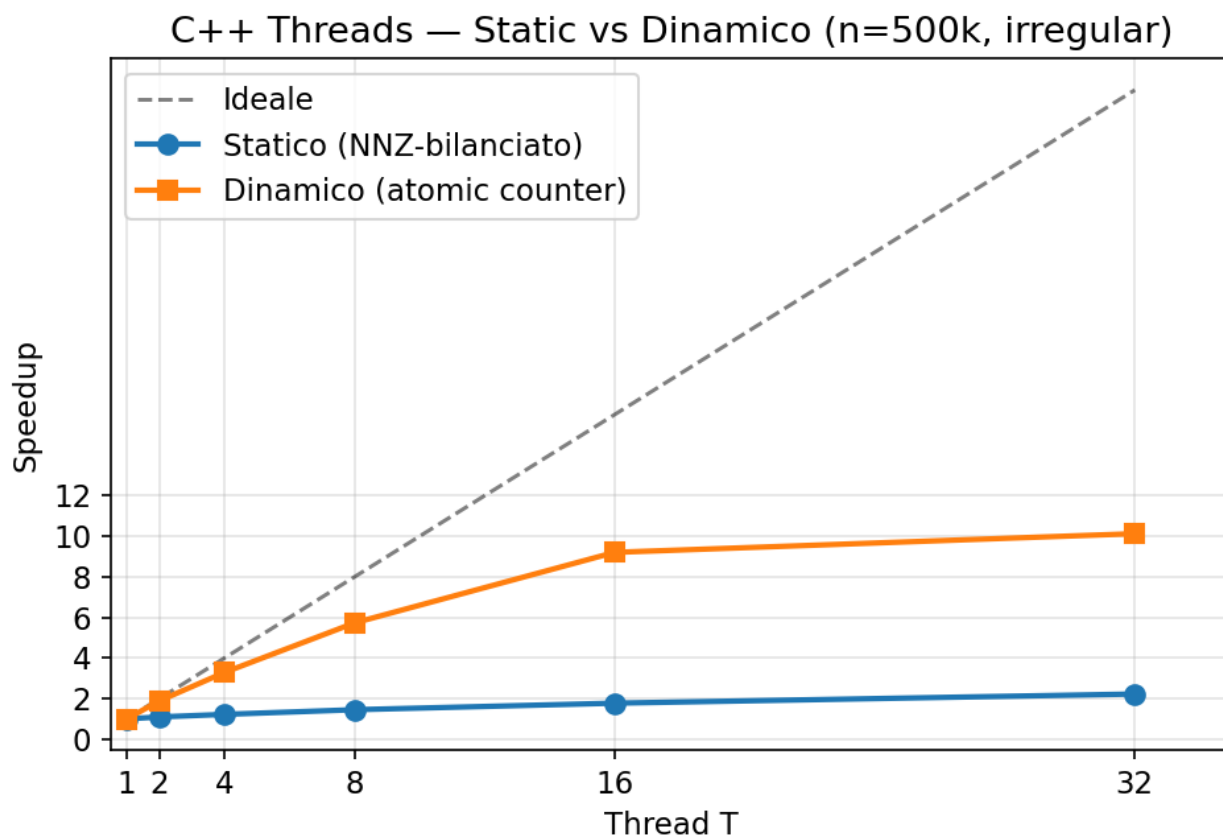


Fig. 1 - Speedup statico vs dinamico (C++ Threads)

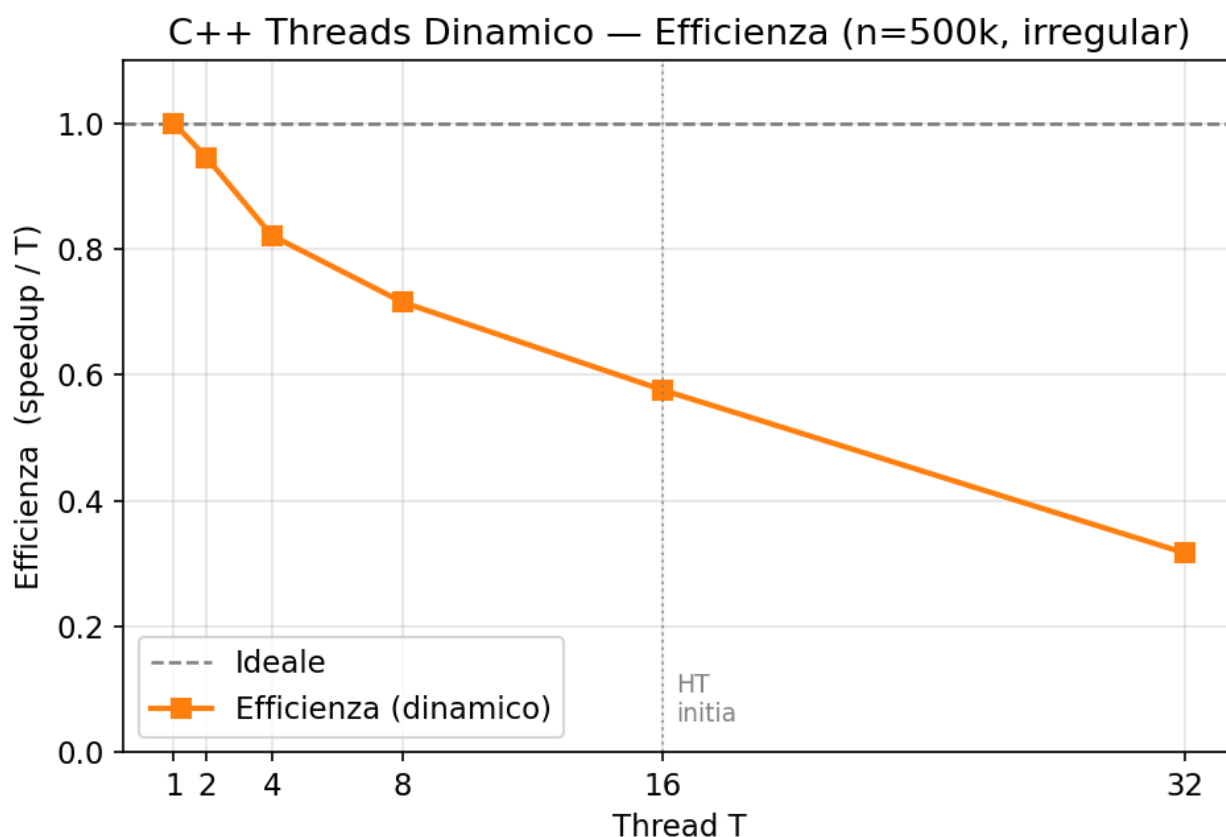


Fig.2 - Efficienza scheduling dinamico

4.2 Granularità dei Task OMP (sweep su K, T=16 fisso)

K	Task/iter	Tempo (s)	Note
16	31 250	102.15	Creazione task serializzata -> **peggio del sequenziale
64	7 812	21.70	Ancora dominato dall'overhead
256	1 953	5.52	Il calcolo inizia a dominare
1024	488	4.83	Ottimale

Analisi. La creazione dei task è serializzata nel blocco `omp single`. Per $n = 500\,000$, $K = 16$ significa 31 250 oggetti task per iterazione x 500 iterazioni = 15.6 M task totali. L'overhead di scheduling (~100-500 ns per task) si accumula fino a >7 s di overhead puro, mascherando il calcolo. A $K=1024$ vengono creati solo 488 task/iterazione; ogni task copre in media 40 NNZ/riga x 1024 righe ~ 41 000 operazioni FP ? overhead di scheduling (~200 ns). Il punto di inflazione è vicino a $K=256$ dove il rapporto calcolo:overhead supera ~100x.

La stessa degradazione appare nello sweep su T a $K=64$ fisso: $T=8$ dà 10.4 s, $T=16$ regredisce a 21.7 s, $T=32$ a 30.5 s. Con 16 consumer, il produttore a singolo thread che crea 7 812 task/iterazione diventa il bottleneck. Questo viene eliminato a $K=1024$ dove la creazione è abbastanza veloce da saturare anche $T=32$.

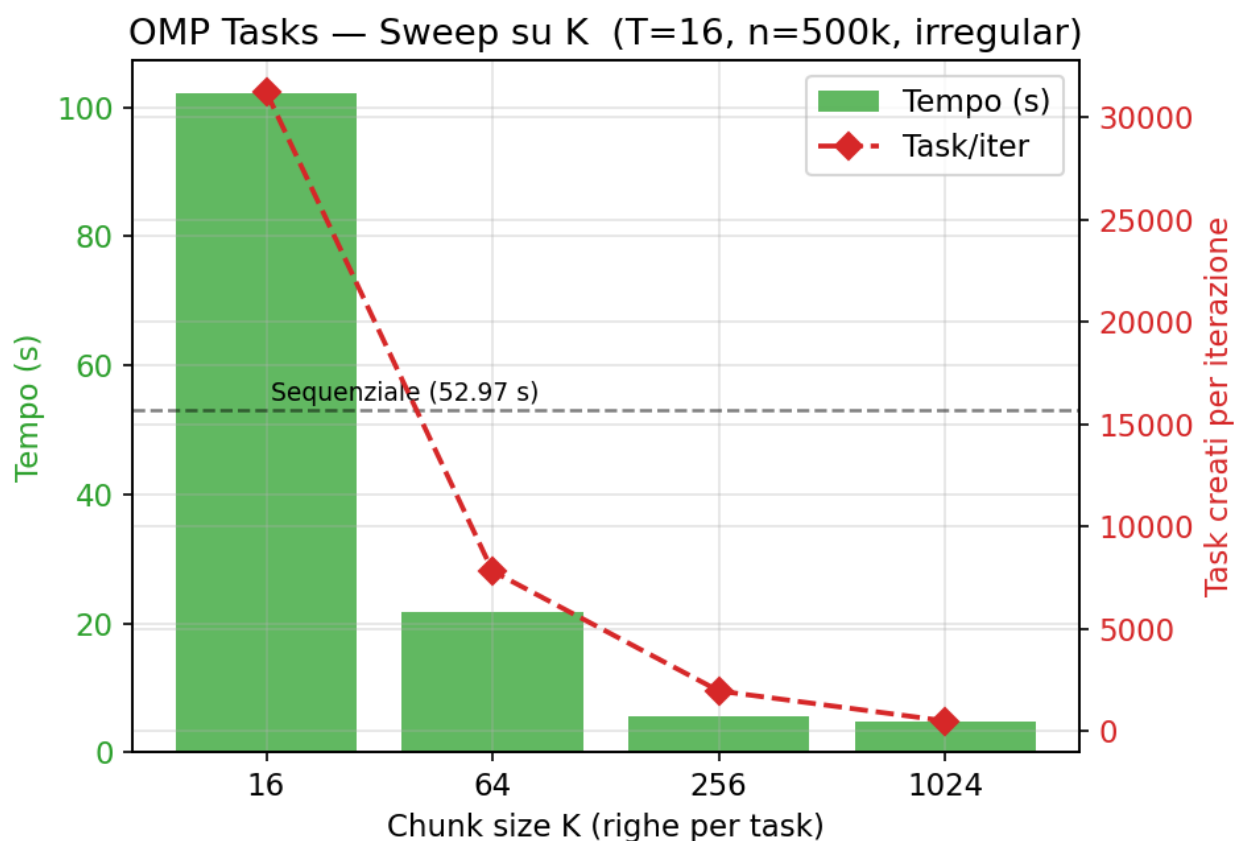


Fig.3 - Sweep su K: tempo e task/iter (OMP Tasks, T=16)

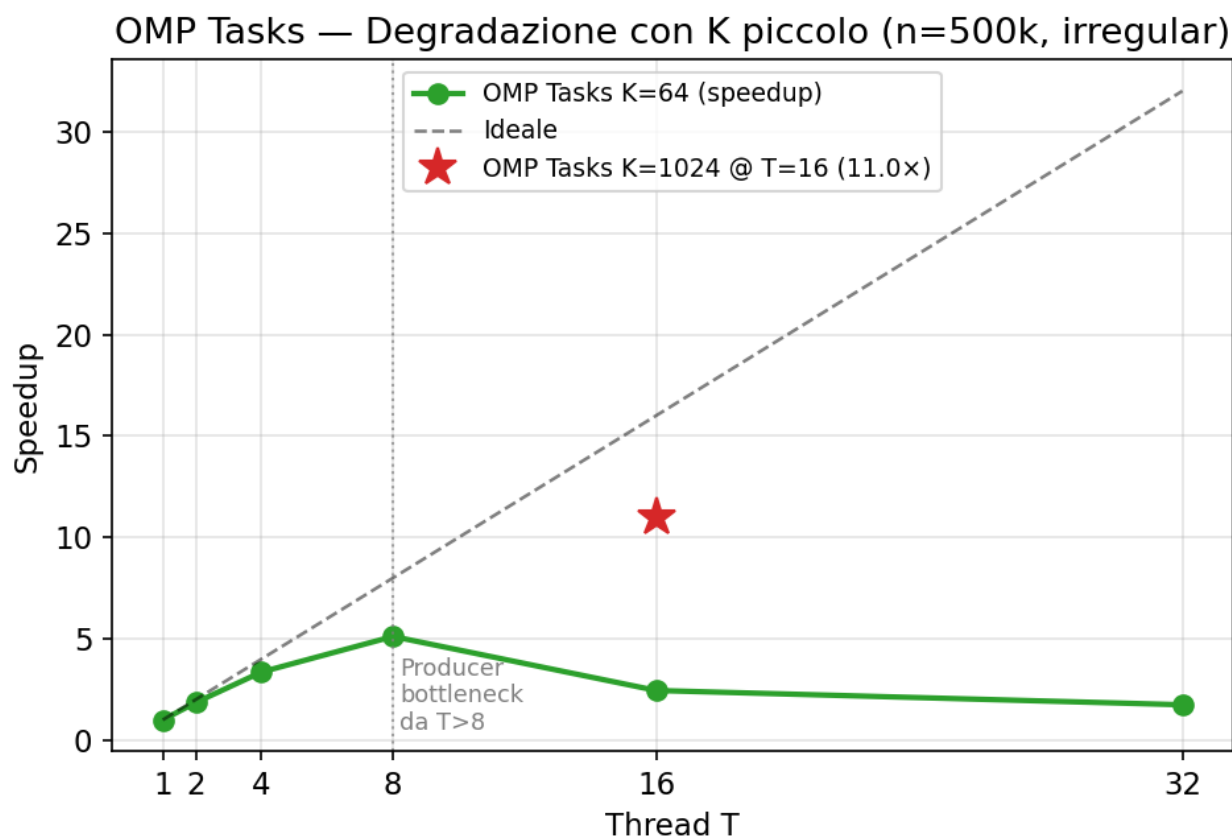


Fig.4 - Degradazione OMP Tasks con K=64 al crescere di T

4.3 Confronto Implementazioni a T=16 (n=500k, irregular)

Implementazione	Tempo (s)	Speedup	Note
Threads Statico	29.85	1.78x	Bilanciamento NNZ rotto dagli epoch
Threads Dinamico (K=64)	5.77	9.21x	Overhead fetch_add atomico
OMP Tasks (K=64)	21.70	2.44x	Bottleneck creazione task
OMP Tasks (K=1024)	4.83	10.97x	K ottimale
OMP Work-Sharing (K=64)	5.02	10.56x	Nessun bottleneck di creazione

OMP Tasks all'K ottimale (4.83 s) supera leggermente OMP Work-Sharing (5.02 s). Questo può essere dovuto al fatto che il runtime task può fare work-stealing granulare dalla coda dei thread, smussando le micro-irregolarità all'interno di un chunk. OMP Work-Sharing fornisce risultati pressoché identici con minor necessità di tuning: evita completamente la sensibilità a K (nessun produttore a singolo thread) e raggiunge un buon bilanciamento tramite il meccanismo di contatore condiviso del runtime.

Confronto implementazioni single-node

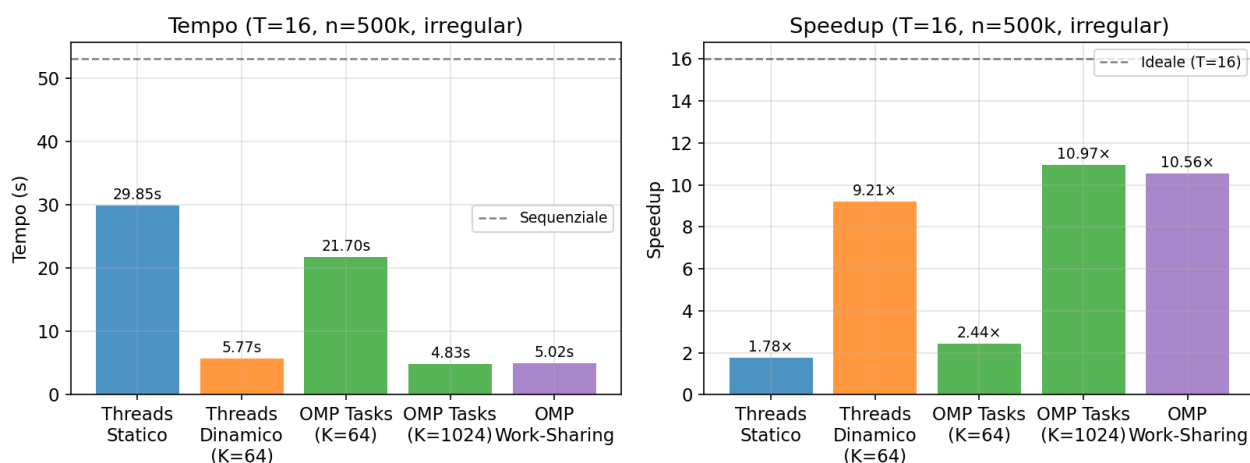


Fig.5 - Confronto implementazioni a T=16

5. Performance MPI+OpenMP

Configurazione: n = 2 000 000, nz = 80 000 000, -m *irregular*. Riferimento sequenziale = **217.93 s**. T = 4 thread per processo per tutto lo strong/weak scaling (salvo diversa indicazione).

5.1 Strong Scaling (dimensione problema fissa)

NP	Nodi	Core totali	Tempo (s)	Speedup vs seq	Efficienza
1	1	4	208.35	1.05x	0.261
2	1	8	180.93	1.20x	0.151
4	1	16	126.95	1.72x	0.107
8	2	32	91.43	2.38x	0.074
16	4	64	57.88	3.76x	0.059
32	8	128	49.49	4.40x	0.034

L'efficienza di strong scaling è bassa: 4.4x su 128 core logici. La causa principale è `MPI_Allgatherv`, che trasferisce $O(n) = 16$ MB di dati ad ogni iterazione indipendentemente dal numero di processi, per un totale

di 16 MB x 500 = 8 GB sull'intero run. Questo è un *volume di comunicazione costante* - non diminuisce all'aumentare di NP - quindi aggiungere processi non riduce il costo di comunicazione.

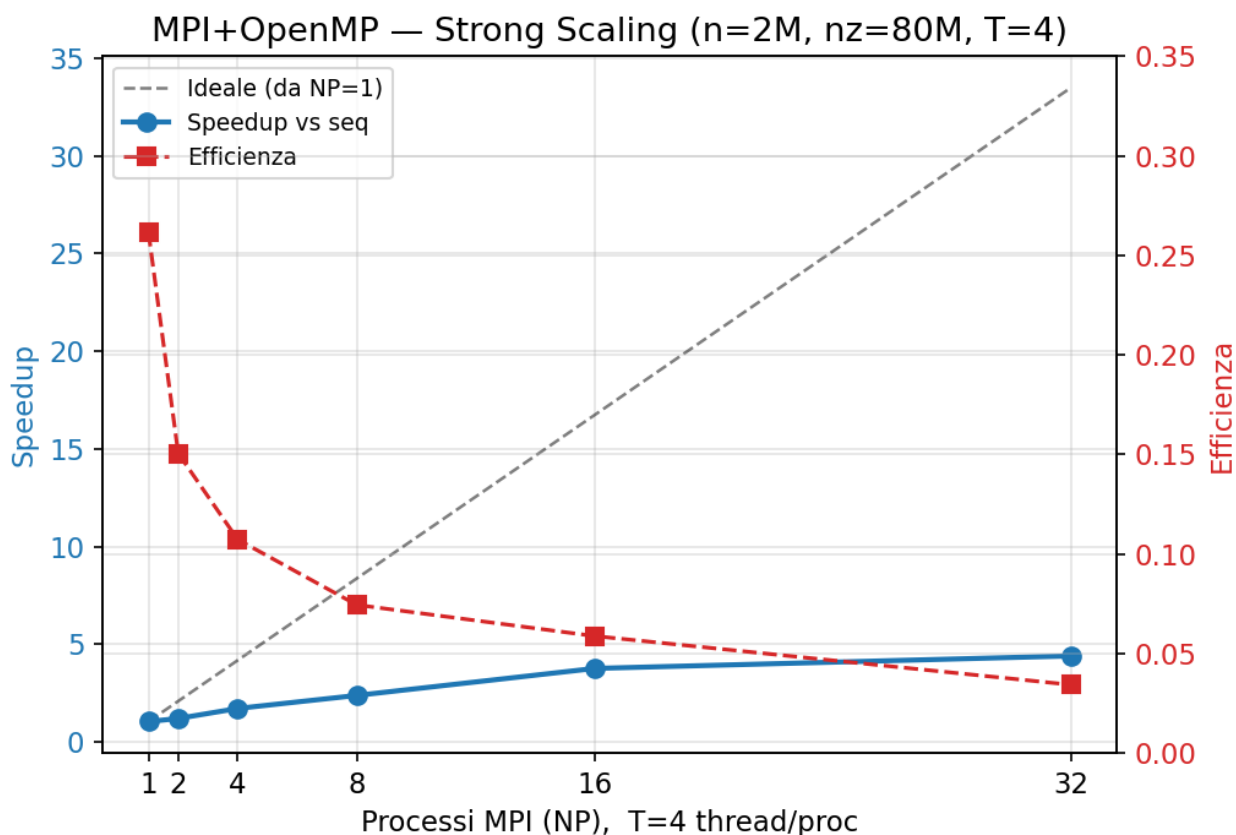


Fig.6 - MPI Strong Scaling: speedup ed efficienza

5.2 Breakdown del Tempo di Esecuzione (Strong Scaling)

NP	Calcolo locale (s)	Comm/Allgather (s)	Allreduce (s)	Epoch (s)	Totale (s)
1	202.6	4.89	0.002	~0	208.4
2	172.7	5.29	~2.9	~0	180.9
4	117.6	9.19	0.13	~0	127.0
8	58.9	32.4	0.07	~0	91.4
16	29.6	19.2	9.1	~0	57.9
32	14.7	22.4	12.4	~0	49.5

Emergono tre fasi al crescere di NP:

- **NP ≤ 4 (intra-nodo):** il calcolo locale domina. La comunicazione (Allgather in memoria condivisa) aggiunge ~9 s di overhead. La latenza dell'Allreduce è trascurabile (<0.1 s). Il costo di transizione epoch è zero - il mapping logico evita ogni movimento di dati.
- **NP = 8 (primo salto inter-nodo):** l'Allgather balza a 32.4 s. La banda effettiva per MPI intra-nodo è ~870 MB/s; al confine inter-nodo scende a ~250 MB/s (stimata da $64 \times 106 \times 8 \text{ B} \times 500 \text{ iter} / 32.4 \text{ s}$). Il calcolo locale si dimezza a 58.9 s. La comunicazione rappresenta ora il **35 %** del tempo totale.
- **NP ≥ 16 (multi-nodo):** sia Allgather che Allreduce crescono. A NP=32 il calcolo locale è solo 14.7 s (29 % del totale), mentre comunicazione (22.4 s) e riduzione (12.4 s) insieme rappresentano il **71 %** del tempo di esecuzione. Aggiungere processi sposta semplicemente il tempo dal calcolo alla comunicazione - rendimenti decrescenti.

Costo transizione epoch = 0 in tutte le configurazioni, confermando che l'Opzione A (mapping logico) è la scelta corretta. L'Opzione B (ridistribuzione `MPI_Alltoallv` ad ogni epoch) aggiungerebbe $20 \times O(n)$ operazioni collettive, stimate a ~100-200 s di overhead aggiuntivo in base alla banda `Allgatherv` misurata.

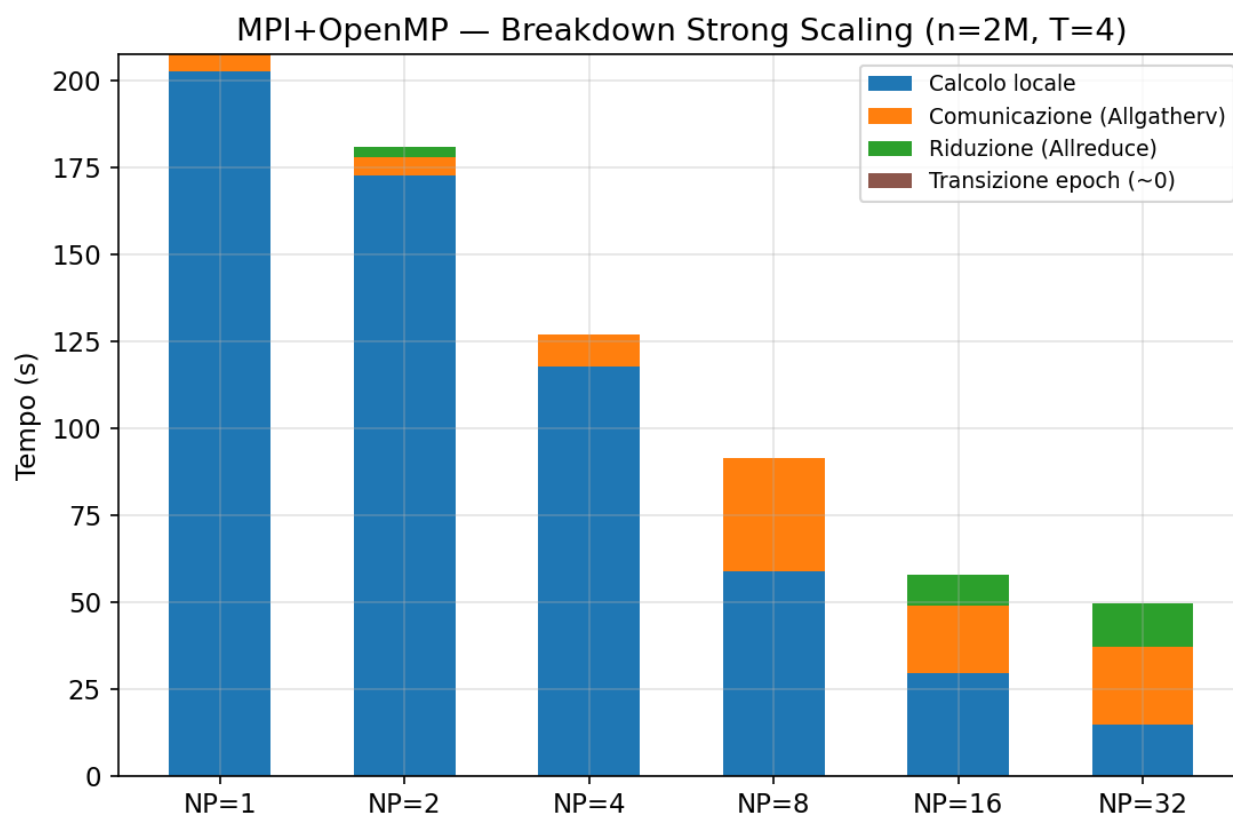


Fig. 7 - Breakdown tempo esecuzione (Strong Scaling)

5.3 Weak Scaling (lavoro costante per processo: ~20M nnz)

NP	Nodi	n	Tempo (s)	Calcolo (s)	Comm (s)	Riduzione (s)
1	1	500k	42.3	41.1	0.97	0.002
2	1	1M	59.6	54.5	2.44	~2.1
4	1	2M	127.6	118.4	9.21	0.009
8	2	4M	194.9	130.1	64.7	0.065
16	4	8M	250.7	135.8	73.2	41.7
32	8	16M	431.4	139.9	179.3	112.1

Il weak scaling è gravemente non ideale. In condizioni di weak scaling perfetto il tempo di esecuzione dovrebbe rimanere costante al crescere proporzionale di NP e n. Il tempo totale cresce invece da 42.3 s a 431.4 s (10.2x) mentre NP va da 1 a 32 (32x).

La ragione è che `MPI_Allgatherv` invia **n_totale** double ad ogni iterazione, e $n_{\text{totale}} = n_{\text{per_proc}} \times \text{NP}$ cresce linearmente con NP. Questo è un limite architetturale fondamentale del design con x replicato: un aumento di 32x in NP raddoppia sia il numero di processi sia n_{totale} (weak scaling), quindi il volume dati dell'`Allgatherv` cresce di 32x. Con banda inter-nodo ~250 MB/s, i 16M x 8 B x 500 iter = 64 GB trasferiti a NP=32 richiedono da soli ~256 s di tempo di rete.

Il calcolo locale cresce solo leggermente (41.1 -> 139.9 s) a causa dell'aumento di n che provoca cache pressure ed effetti NUMA, confermando che il calcolo non è il bottleneck.

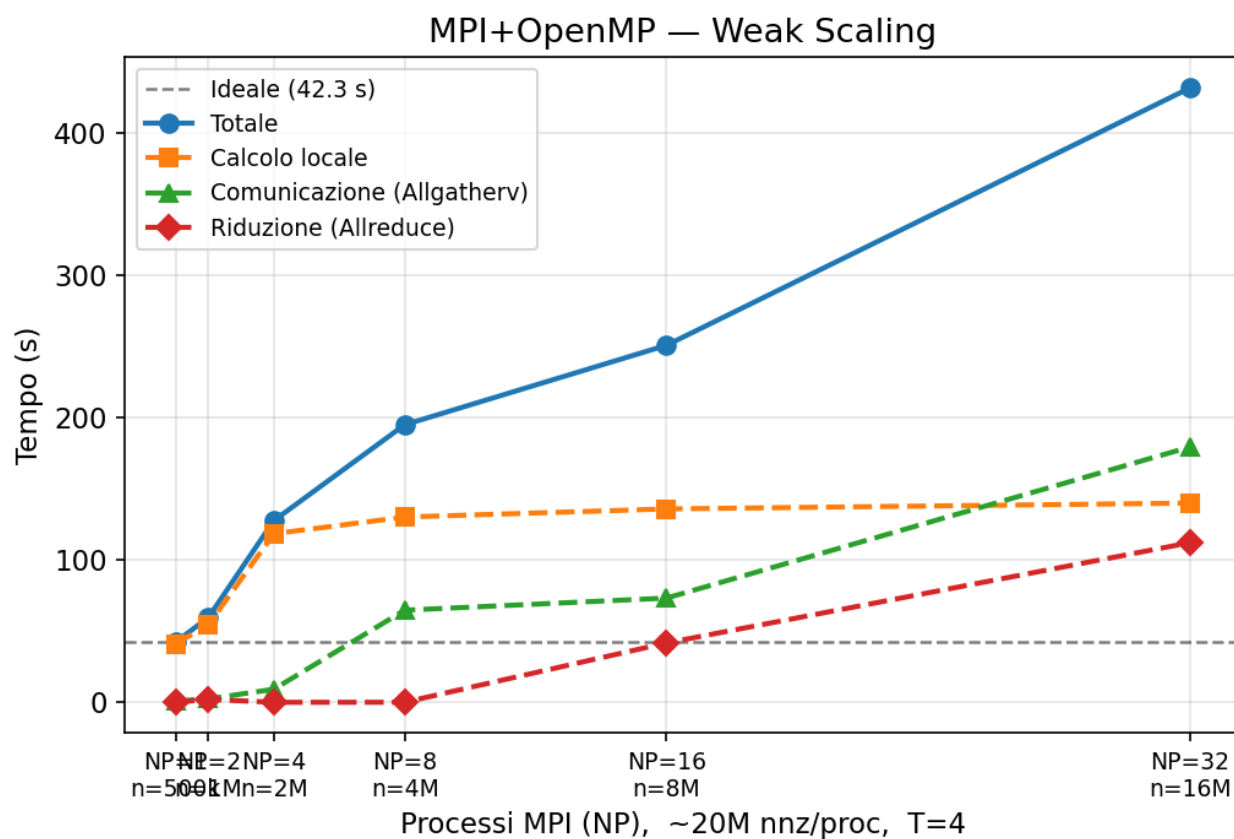


Fig.8 - MPI Weak Scaling: totale e componenti

5.4 Interazione NP x T (n=2M, nz=80M)

Su problema fisso, diverse combinazioni (NP, T) con lo stesso numero totale di core logici producono performance diverse perché influenzano il rapporto calcolo/comunicazione:

Core totali	NP	T/proc	Nodi	Tempo (s)	Calcolo (s)	Comm+Rid (s)
16	1	16	1	208.1	202.4	4.89
16	2	8	1	180.9	171.5	9.25
16	4	4	1	127.1	117.6	9.35
16	8	2	1	90.6	79.6	10.96
16	16	1	1	61.2	41.1	20.1
32	2	16	2	116.1	101.1	15.3
32	4	8	2	107.0	85.7	21.4
32	8	4	2	91.3	58.7	32.5
32	16	2	2	64.4	39.3	25.0
32	32	1	2	50.1	20.2	29.9
64	4	16	4	70.0	50.6	18.9
64	8	8	4	71.6	42.7	29.0
64	16	4	4	57.9	29.6	28.3
64	32	2	4	50.0	20.0	29.6
64	64	1	4	39.5	10.2	29.5
128	8	16	8	48.2	25.9	22.2
128	16	8	8	59.4	21.9	37.4
128	32	4	8	49.5	14.7	34.8

128	64	2	8	41.8	10.1	31.6
128	128	1	8	34.8	5.1	29.7

Osservazioni principali:

- **Più processi, meno thread per processo è generalmente meglio** a parità di core totali. A 16 core totali, np=16 t=1 (61.2 s) batte np=1 t=16 (208.1 s). La ragione: più processi MPI = slice n per processo più piccola = calcolo OpenMP locale più veloce. Il volume Allgatherv ($O(n_{\text{totale}})$) è identico in entrambi i casi.
- **La frazione di calcolo svanisce ad alto NP.** A NP=128, T=1 (8 nodi): calcolo locale = 5.1 s vs totale = 34.8 s. Solo il **14.7 %** del tempo è speso in calcolo; il restante 85.3 % è comunicazione. Aggiungere ulteriori risorse non fornirebbe alcun beneficio ulteriore poiché il calcolo è già trascurabile.
- **A nodi fissi (es. 8 nodi), la configurazione ottimale è NP=128 T=1** (34.8 s) invece di NP=8 T=16 (48.2 s). Questo è controintuitivo: più processi MPI con rank single-threaded superano meno processi con rank multi-threaded. La spiegazione: la comunicazione MPI intra-nodo (memoria condivisa) è molto veloce, quindi np=16 per nodo x 8 nodi = 128 processi a T=1 aggiunge ~29.7 s di overhead Allgatherv (stesso volume dati del caso T=16) ma riduce il calcolo da 25.9 s a 5.1 s.
- **MPI_Allreduce diventa significativo ad alto NP** (12.4 s a NP=32, 14.6 s a NP=64, 10.9 s a NP=128). Questo costo di latenza $O(\log P)$ cresce perché la latenza della riduzione attraverso molti nodi non è trascurabile quando si accumulano 500 riduzioni.

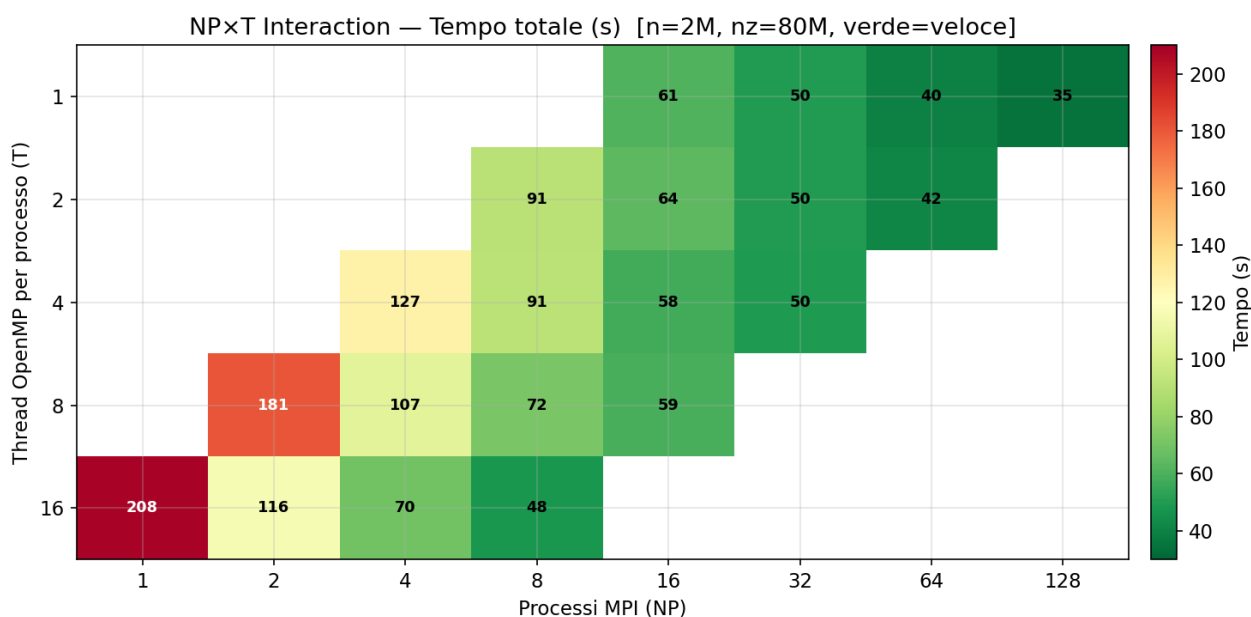


Fig.9 - Heatmap NP x T: tempo totale (s)

6. Analisi dei Bottleneck e Limiti di Scalabilità

Bottleneck primario: MPI_Allgatherv - $O(n)$ per iterazione. Ogni iterazione richiede la ricostruzione di x su tutti i rank raccogliendo le slice normalizzate di y_local. Il volume totale di dati è n double indipendentemente da NP. Con banda intra-nodo ~870 MB/s e inter-nodo ~250 MB/s, gli 8 GB totali trasferiti a n=2M sono il costo dominante a $NP \geq 8$. Questo è un limite intrinseco del design con x replicato, non un problema di tuning.

L'alternativa (halo exchange - ogni rank preleva solo le colonne di x necessarie per le sue righe) ridurrebbe la comunicazione per pattern di sparsità strutturati, ma per indici di colonna casuali irregolari il working set delle voci di x per riga non è correlato ai boundary dei rank, richiedendo una comunicazione all-to-all non uniforme che potrebbe essere ancora più lenta in pratica.

Bottleneck secondario: MPI_Allreduce - latenza $O(\log P)$ per iterazione. 500 operazioni Allreduce a NP=32 costano 12.4 s = 24.8 ms/chiamata. A NP=128 questo è 10.9 s = 21.8 ms/chiamata. La riduzione riguarda un singolo double; la latenza è dominata dalla profondità dell'albero e dai round-trip di rete piuttosto che dalla banda. Diventa una frazione significativa (25 %) del tempo totale a NP=32.

Bottleneck di sbilanciamento (single-node): rotazione degli epoch. Il partizionamento statico NNZ-bilanciato fallisce su input irregolare perché il mapping logico->fisico ruota ad ogni epoch, ridistribuendo le righe pesanti tra i thread. Lo scheduling dinamico (contatore atomico) elimina questo problema al costo di un LOCK XADD per chunk di K righe. A K=64 l'overhead è ~3.5 % del tempo di calcolo per 16 thread.

Saturazione della banda di memoria. A n=2M con 4 thread OMP, il calcolo locale per processo MPI è ~202.6 s contro il teorico 4x speedup rispetto al sequenziale. Questo suggerisce saturazione della banda di memoria: gli 80M NNZ di valori double (~960 MB) non entrano in nessun livello di cache; lo SpMV è completamente bandwidth-bound. L'hyperthreading (T=32) fornisce solo un miglioramento marginale (4.58 s vs 5.02 s a T=16 per OMP WS su n=500k), coerente con la condivisione della banda tra core logici.

7. Conclusioni

Metrica	Risultato
Miglior single-node (T=16, n=500k)	OMP Tasks K=1024: 10.97x speedup
Strong scaling (NP=32, T=4, n=2M)	4.40x speedup su 128 core
Weak scaling (NP=32, T=4)	rallentamento 4.1x (dominato dalla comunicazione)
Overhead transizione epoch	~0 (mapping logico, nessuna ridistribuzione)

Le implementazioni single-node dimostrano che la distribuzione dinamica del lavoro è essenziale per SpMV irregolare: il partizionamento statico NNZ-bilanciato viene rotto dalla rotazione degli epoch, mentre il chunking dinamico raggiunge scaling quasi lineare fino al numero di core fisici. OMP Work-Sharing offre il miglior equilibrio tra performance e robustezza (nessun tuning di K richiesto); OMP Tasks all'K ottimale (1024) è leggermente migliore ma sensibile al parametro di granularità.

L'implementazione distribuita MPI+OpenMP è fondamentalmente limitata dall'Allgather $O(n)$ per iterazione. Sebbene la strategia di mapping logico elimini completamente il costo di ridistribuzione, lo schema con x replicato richiede di trasmettere n double ad ogni iterazione. A n=2M e 8 nodi, la comunicazione consuma il 71 % del tempo di esecuzione. Il soffitto pratico di scalabilità per questo algoritmo con x replicato è circa NP=8-NP=16 (4x speedup), oltre il quale ogni processo aggiunto riduce il calcolo meno di quanto aggiunga in overhead di comunicazione. A 128 core totali la configurazione ottimale è NP=128 T=1 (34.8 s, 6.26x speedup vs sequenziale), dove il calcolo è essenzialmente gratuito e l'intero costo è comunicazione.

Un'implementazione scalabile richiederebbe ridurre la comunicazione per iterazione da $O(n)$ a $O(n/P)$ per rank, il che implica partizionare x o adottare un approccio gather-scatter che scambi solo le voci di x effettivamente necessarie per ogni rank - scambiando volume di comunicazione per complessità di comunicazione. Questo viene lasciato come lavoro futuro.